

RapidChain

Introduction & Problem Definition

RapidChain is a public blockchain protocol designed to overcome the performance and scalability limitations of its predecessors through a technique known as **full sharding**. The protocol operates on a network model of up to 4,000 nodes, assuming all participants have roughly equivalent computational resources and without the need for a trusted source of initial randomness. Its primary goal is to significantly increase transaction throughput and reduce latency by dividing the overheads of processing transactions among multiple, smaller groups of nodes called committees, which work in parallel.

Comparison with Other Consensus Models

To understand RapidChain's contribution, it's useful to compare it to other consensus architectures:

Consensus Model	How it Works	Example	Key Limitation
Nakamoto Consensus	Every node validates every transaction. A single, monolithic ledger.	Bitcoin	Not scalable; low throughput, high latency, and high energy consumption.
Committee-Based	A smaller, select group of nodes (a committee) handles consensus.	ByzCoin, Solida	More efficient but can still be a bottleneck. Prone to issues like biased randomness or reliance on a trusted setup.
Sharding-Based	Multiple committees work in parallel on separate ledger partitions (shards).	Elastico, OmniLedger	Earlier versions still suffered from "gossip-to-all" communication bottlenecks, had lower fault tolerance (1/4), and often required a trusted setup.

RapidChain builds upon the sharding model but aims to be the first **fully sharding** protocol that is **1/3 resilient** and requires **no trusted setup**, solving the key

limitations of prior work.

RapidChain's architecture is built on three main components that manage the network's lifecycle from inception to daily operation and evolution.

1. Bootstrapping (One-Time Setup)

This is the initial, one-time process that runs when the network is first created. Its purpose is to securely form the initial committees without needing a trusted central party.

- **Process:** It uses a "sampler graph" and a multi-level election network to ensure that even with a malicious minority of nodes, the resulting committees will have an honest majority.
- **Outcome:** This process establishes a secure **Reference Committee (CR)** and the initial set of **Sharding Committees**.
- **Cost:** This security comes at a high initial cost. In experiments, the bootstrapping for a 4,000-node network took **18.5 hours** and consumed significant bandwidth. This cost is amortized as it only happens once.

2. Consensus (Day-to-Day Operation)

This is the component responsible for the day-to-day processing of transactions. The vast majority of the protocol's time is spent in this phase.

- **Process:** When transactions are submitted to the network, they are routed to the appropriate Sharding Committee. The members of that committee then run an intra-committee consensus protocol to agree on and validate a block of these transactions.
- **Outcome:** Validated blocks are appended to that committee's specific ledger, or "shard."

3. Reconfiguration (Network Management)

This component manages changes in the network, such as nodes joining or leaving. It is a short phase that runs at the end of every long epoch (e.g., once a day).

- **Process:** This phase is driven by the Reference Committee. It generates fresh randomness for the next epoch and securely updates the committee memberships.
- **Outcome:** The network adapts to churn (changes in participation) without compromising its security guarantees.

Committee Roles

- **Reference Committee (CR):** A special committee with two key responsibilities:
 1. **Randomness Generation:** It runs a Distributed Random Generation (DRG) protocol using Verifiable Secret Sharing (VSS) to produce unbiased randomness for each epoch. This is vital for secure leader selection and puzzle generation.
 2. **Reconfiguration Management:** It checks the Proof-of-Work solutions from all nodes and manages the reconfiguration process.
- **Sharding Committees:** The workhorses of the network. They operate in parallel, each maintaining its own shard of the blockchain, enabling high throughput.

The consensus within each sharding committee (intra-committee consensus) is highly optimized for performance and security.

The 4-Round Synchronous Consensus Protocol

When a leader is selected for an iteration, it proposes a block. To agree on this block, the committee engages in a four-round protocol:

1. **Propose:** The leader creates a block header (containing a Merkle root of transactions) and gossips it to the committee.
2. **Echo:** Members re-broadcast the header they received. This step is crucial for detecting a cheating leader who might have sent different headers to different members (an act known as equivocation).
3. **Pending:** If a node detects conflicting headers from the Echo stage, it knows the leader is corrupt and votes for a special "pending" value.

4. **Accept:** If a node receives a sufficient number of matching echoes for a single header, it accepts the block as valid and broadcasts its acceptance note.

Pipelining for Performance

A key challenge in consensus is that a malicious leader can cause a block to get "stuck" in a pending state. Simply waiting for the next leader to re-propose this single block would cut the network's throughput in half.

- **Pipeline Solution:** RapidChain solves this by allowing the leader of a new iteration to work on two tasks at once:
 1. Propose a **new block** of transactions.
 2. Simultaneously re-propose the headers of any **old, pending blocks**.
- **Impact:** This pipelining mechanism ensures the network is always making forward progress on new transactions, even while resolving past issues. This optimization was shown to **improve throughput by about 1.4x**.

Communication Model: A Hybrid Approach

RapidChain strategically uses two different network models:

- **Synchronous:** For the critical **intra-committee consensus** on block headers, the protocol uses a synchronous model. This means there is a fixed maximum time for message delivery (e.g., 600 ms).
 - **Pro:** This allows for higher fault tolerance (tolerating $< 1/2$ faulty nodes within a committee).
 - **Con:** It is **not responsive**, meaning its speed is fixed and cannot adapt to a faster network.
- **Partially Synchronous:** For all other parts of the protocol (like routing), it uses a partially synchronous model with **optimistic, exponentially-increasing timeouts**.
 - **Benefit:** This makes most of the system highly **responsive**, achieving low latency while still being reliable.

Inter-Committee Routing: Kademlia

A major flaw in earlier sharding protocols was "gossip-to-all," where messages were broadcast across the entire network, creating a bottleneck. RapidChain avoids this using a Kademlia-inspired routing mechanism for communication **between committees**.

- **How it Works:** Each committee has a unique ID. Instead of knowing every other committee, a committee maintains a small routing table of other committees at logarithmic "distances." To send a message, it forwards it to the known committee closest to the destination, which repeats the process.
- **Result:** This enables message routing in a very efficient **logarithmic** ($\log n$) **number of steps**.

Cross-Shard Transactions

When a transaction's inputs and outputs reside on different shards, the process is handled automatically by the committees.

1. The transaction is routed to the correct "output committee" using Kademlia.
2. The output committee identifies the "input committees" that hold the funds (UTXOs) being spent.
3. It then sends verification requests to these input committees.
4. Crucially, the output committee **batches** many requests for the same input committee into a single message, drastically reducing communication overhead.

RapidChain includes specific mechanisms to allow new nodes to join securely and to protect the system from long-term adversarial strategies.

Joining the Network (During Reconfiguration)

A new node must follow a strict process during the brief reconfiguration phase at the end of an epoch:

1. **Wait for Randomness:** The process begins only when the Reference Committee (CR) reveals the fresh randomness (r_{i+1}) for the next epoch.
2. **Solve the Puzzle:** The node has a window of **roughly 10 minutes** to solve the PoW puzzle, which is based on this new randomness. This prevents pre-

computation by an adversary.

3. **Submit Solution:** The solution must be submitted to the Reference Committee before the current epoch officially ends.
4. **Initialization:** Once accepted and assigned to a committee, a new node only needs to download the current set of **unspent transactions (UTXOs)** from its committee members, not the entire historical ledger.

Offline Proof-of-Work (PoW)

To prevent Sybil attacks, nodes must perform Proof-of-Work. However, this is done efficiently:

- The expensive PoW calculations are performed **offline**, meaning they do not block or delay the critical path of transaction processing. A node solves its puzzle during the brief reconfiguration phase without halting the entire system.

RapidChain's performance relies heavily on how it manages communication and transactions that span different shards.

Security Against Churn: The Cuckoo Rule

A significant threat is that an adversary could strategically have nodes leave and rejoin the network over time to eventually gain control of a sharding committee. Re-electing all committees every epoch is too expensive.

- **Cuckoo Rule Solution:** RapidChain uses the Cuckoo Rule to defend against this. During each reconfiguration, only a **small, constant number of nodes** are moved ("cuckoo'ed") from their committees and reassigned to new random ones.
- **Benefit:** This constant, unpredictable shuffling provably maintains the security and balance of all committees over time without incurring the high cost of a full network-wide re-election.

While groundbreaking, the RapidChain protocol has several known challenges and trade-offs inherent in its design. Since its publication, other protocols have emerged that address these limitations in different ways.

Identified Challenges of RapidChain

- **Lack of Responsiveness:** The reliance on synchronous consensus for security means its core processing speed is fixed and cannot adapt to faster network conditions.
- **High Cost of Bootstrapping:** The initial one-time setup is extremely long and resource-intensive, presenting a significant barrier to launching a new network.
- **Protocol Complexity:** The design integrates many advanced components (sampler graphs, IDA, Kademlia, Cuckoo rule), making it complex to implement and maintain correctly.
- **Security Model Assumptions:** The protocol's security proofs rely on specific assumptions that may not hold in all real-world scenarios:
 - **Slowly-Adaptive Adversary:** Assumes an adversary can only corrupt nodes between epochs, not during them.
 - **Bounded Churn:** Assumes only a small, constant number of nodes join or leave per epoch.
 - **Equivalent Computational Resources:** Assumes all participants have similar computing power.

Modern Alternatives and Future Directions

- **Avalanche:** Introduces a metastable, leaderless consensus protocol that is highly responsive and lightweight. It avoids complex bootstrapping and reconfiguration phases entirely, offering a simpler and more adaptive approach to consensus.
 - *Reference: [Team Rocket. \(2018\). Snowflake to Avalanche: A Novel Metastable Consensus Protocol Family for Cryptocurrencies.](#)*
- **Algorand:** Its Pure Proof-of-Stake (PPoS) protocol is designed to be secure against fully-adaptive adversaries. It uses a Verifiable Random Function (VRF) for cryptographic self-selection of committees, which is extremely fast and eliminates the need for a costly bootstrapping process.

- *Reference: Gilad, Y., et al. (2017). Algorand: Scaling Byzantine Agreements for Cryptocurrencies. SOSP '17.*
- **HotStuff:** This leader-based BFT protocol is designed for partial synchrony, making it responsive to network conditions. Its communication flow is often simpler than all-to-all voting within a committee, addressing protocol complexity. It formed the basis for Diem (formerly Libra).
 - *Reference: Yin, M., et al. (2019). HotStuff: BFT Consensus in the Lens of Blockchain.*
- **Ethereum 2.0 (The Merge):** Ethereum's protocol, Gasper, is also responsive and uses VRFs to defend against adaptive adversaries. Its sharding roadmap (Danksharding) presents a more advanced and integrated model that aims to simplify cross-shard communication and data availability challenges faced by earlier models like RapidChain.
 - *Reference: Buterin, V., & Griffith, V. (2019). Casper the Friendly Finality Gadget. arXiv:1710.09437.*

Thank you for your attention.

This presentation was an overview of the paper:

RapidChain: Scaling Blockchain via Full Sharding

Video presentation available at:

RapidChain-Presentation

Presented by:

Alireza Karimi

810101492

Computer Engineering

University of Tehran